System Design: the Forma company app platform, the investor's edition

The same design told for the founder presenting it and a technical investor reading it alone: eight questions, each answered in a sentence, then how it works, what it trades away, and what to check first.

Author	Victor Villacis
Date	2026-10-23
Status	Proposed
Edition	investor, derived from `design.md` at the phase 7 gate commit
Read first	`design.md` for the detail

Contents

Summary in plain words
1. What it is and why now
2. How it works: the system in two pictures
3. Why the second app is the proof
4. What keeps organizations apart
5. What a number means
6. What it costs and how one founder runs it
7. What we do not know yet and what to do first
8. The map to the record

> This edition is for a partner with an engineering background weighing, in one sitting, whether the platform works, what it rests on, and what is still unproven. It summarises the record, the full design document, and adds nothing to it. A few labels survive so that a reader can find the detail there: D1 to D5 are decisions, G1 to G7 goals, N1 to N12 quality requirements, A1 to A18 assumptions, TH1 to TH29 threat rows, RK1 to RK11 risks, and a plain number such as "section 11.2" is a section of the record. Everything else is said in words.

Summary in plain words

Forma is a company app platform: a place where a member of a team describes a small app over the company's own imported data, a model proposes it, a colleague reviews and publishes it, and the team runs it, saves its work in it, and can trust every number it shows, because each one carries a receipt, the server's record of exactly which files, versions, parameters and policy produced it. It replaces the work that today lives in spreadsheets, scripts and chat tabs. It is built on Forma 3.0.0, the founder's own open-source web framework, released on 2026-10-20.

Three terms carry the design. A *publication record* is the row the platform writes when a reviewer approves an app: it binds the exact definition, the shape of its inputs, the schema of any records it keeps and the versions of the components that render it, so that what runs is what was approved and nothing else. A *record command* is the only way a value written back by a form changes: a declared operation carrying the version the writer last saw and a retry key, committed as one database transaction, so that a retry cannot double it and a stale edit cannot silently overwrite. A *role* here is a deployable service, not a permission: the same program runs as four services, each with its own credentials, so that the one that handles model output cannot publish, and the one that publishes cannot read a record.

The design goals are **generality** (one typed app definition, rendered by the platform's own components), **governed persistence** (the publication record and saved work), **correct and traceable data** (receipts and record commands), **isolation** (the organization's id in every key, enforced by the database) and **honest AI** (labels, abstention, validation). The four roles are the *workspace server*, the one place that decides what a member may see and do; the *control plane*, the only writer of publication records; the *data service*, which computes every number and commits every record; and the *workers*, which parse uploaded files in a sandbox and alone talk to the model provider. They run over two stores: a relational database for publications, saved work and records, and immutable column files for the evidence, queried by an embedded analytical engine. The design optimises for isolation and generality, nothing of one organization reaching another and two different apps through one path with no deploy between them, at the expense of expressiveness, since the vocabulary of views is closed and a new widget is a software release, and of generation latency, since every proposal runs as a background job.

The design point is ten organizations, 100 seats, and at the busiest moment 60 query operations a second, where a query operation is one view of an app refreshing and an app has up to six linked views, for about $1,150 a month. Its first proof is the Northstar demo, two apps on one build: a security investigation and a finance review queue with a decision form. Public pages, apps inside an assistant, live connectors, external actions and custom widgets are deferred.

The honest part: this is a design, status Proposed. The rendering is prototyped on Forma 3.0.0 as shipped; the rest is proposed. Every row of data is synthetic, there is no customer, and one founder builds and runs it. Four doors are one-way, the typed definition (D1), the publication record (D2), pooled tenancy (D3) and the two-store split (D4); the rest can be reversed.

1. What it is and why now

The question. What is Forma, and why build it now?

The answer. Forma turns the work a team does today in spreadsheets, scripts and chat tabs into small, governed apps over the company's own data; it is built now because the framework it needs shipped on 2026-10-20, and because the gaps that Y Combinator's request for "small software" names are exactly the gaps the founder's prototypes cannot close. That is a founder's reason; whether a team switches this quarter is the question the record cannot yet answer, and it says so.

How it works. Three things exist today. Forma 3.0.0 is a compiler, a server renderer written in Rust, and a browser runtime; by design it has no identity, no authorization, no storage, no background jobs and no notion of a tenant, so that those can be designed once, here. Three older prototypes on Forma 2.x are donors: one generates and indexes UI modules from a request, one routes an intent to a widget on a canvas, one extracts that canvas as a library. And design A, the predecessor to this record, was a security-first platform for a fictional insurer in which the model wrote markup. This record changes direction on one ruling: the platform is industry-agnostic, and the model proposes a typed definition of an app, which is data, not code, that the platform's own compiled components render; the markup path becomes a later tier for custom widgets. The record is governed by a product definition, the founder's own 25-page document, cited below as "the definition".

What breaks today is best seen as a mechanism. Northstar is a fictional company; Maya is one of its security engineers; the data is synthetic.

1. Maya exports ten minutes of connection events for one host, `build-worker-17`, to a CSV: rows only, no record of which version of the source they came from, and no statement of whether the `bytes` column is per event or cumulative.
2. She matches destination addresses against an inventory with a worksheet lookup. One address has both an exact entry and a network-range entry: a lookup that stops at the first match can take the range over the exact entry, and one that does not stop can count the connection twice. The rule, exact first and once only, lives in her head.
3. She sums 1,140,000 bytes and divides. Over the 5 minutes the events span that is 228,000 bytes a minute; over the 10-minute window she asked about it is 114,000. The sheet shows a number without its denominator.
4. She asks a chat model what the 900,000 unmatched bytes to `198.51.100.7` mean. The pasted rows leave the company's access control; the answer cites no query and is gone when the tab closes.
5. She emails her reviewer a verdict and a screenshot, not the time window, the matching rule or the inventory version. A colleague investigating another host repeats steps 1 to 4 from scratch.
6. Finance repeats all of this with money, and the decision is an overwritable cell with no version, no author and no history; chapter 5 tells that story once, with the fix.

The prototypes do not close this: one serves a single person on the local machine until authentication exists, a dataset is at most 2,000 rows by 32 columns, and none has an organization, saved work, a writable form or a publication policy.

Why now. Y Combinator's Fall 2026 Requests for Startups include "A Cloud for Small Software" by Pete Koomen: "Small software should be as easy to share with your colleagues as a Google Doc." The definition calls Forma's alignment with it "our interpretation"; the request prescribes no app and promises no funding. The constraints are a startup's: one founder builds and runs it; no customer, date, team or budget is supplied; the YC demo is the first audience; the data is synthetic and only measured numbers are called facts. Sign-in is provided by fAuth, an identity service that is a second product of the same founder, live at its own address and past its viability check; the platform uses only its token behind an adapter, so that another provider could replace it.

Positioning. Like Retool, Forma serves internal apps over company data with roles, sign-in and an audit log. Unlike Retool, where an app is code the platform trusts, a Forma app is a typed definition the platform validates and renders, so that the model never ships code and what the reviewer approves is what runs.

The record weighs four more, Airtable, Glide, Claude artifacts and JSON-to-UI libraries, and finds that none pins a typed definition to a publication record with a receipt on every number (section 12.2).

The trade-off. Isolation and generality over expressiveness and the speed of generation: an app that needs a widget the vocabulary lacks waits for a release, and a proposal takes as long as a background job takes.

What to check first. Whether the six mechanisms above describe a team you know. The seventh goal, "cheaper than the spreadsheet it replaces", is marked fuzzy: it can only be measured against a pilot baseline that does not yet exist (section 3.1).

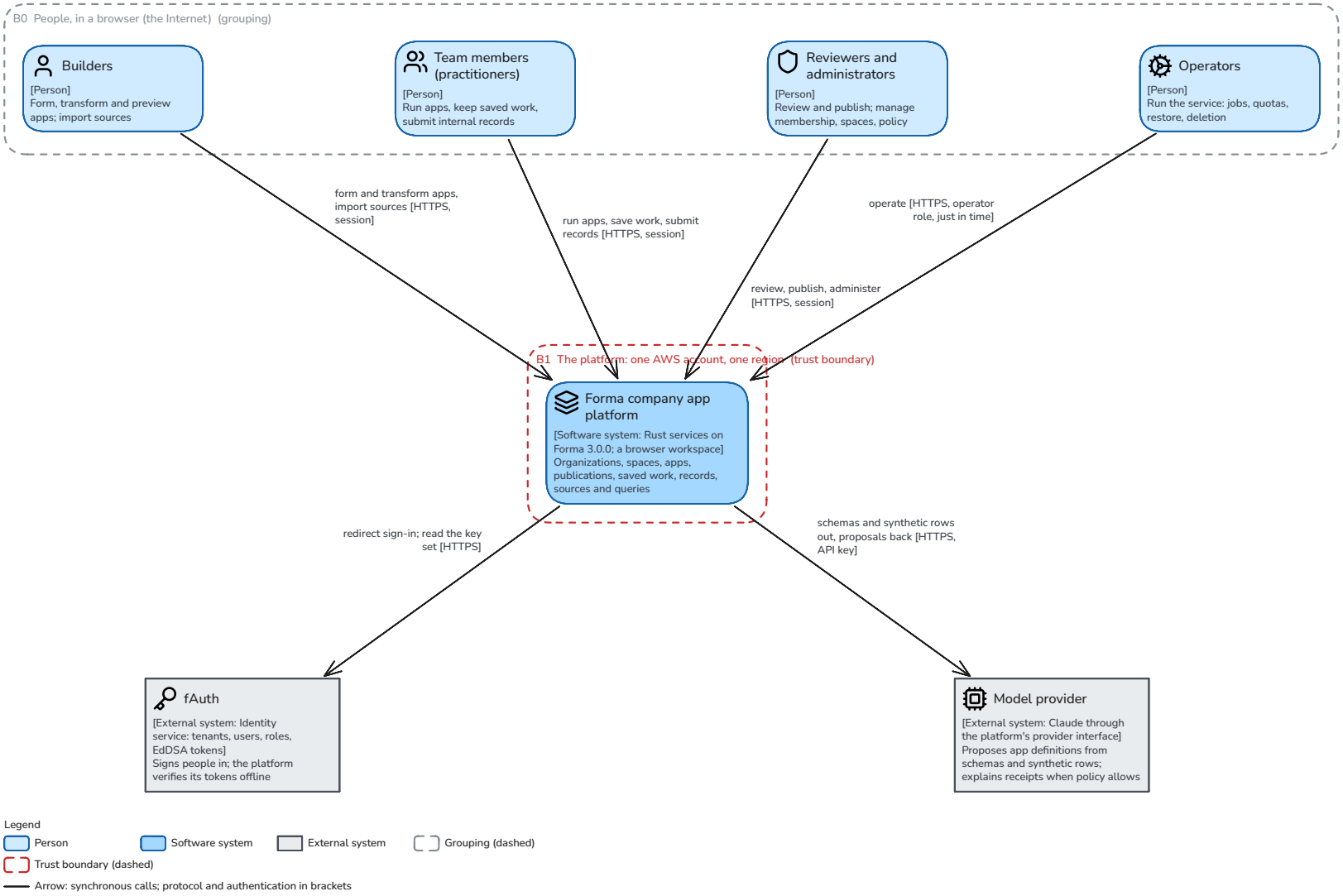
2. How it works: the system in two pictures

The question. What are the parts, and how does a request travel through them?

The answer. One program in four roles, each deployed with its own credentials, over two stores, with one place that decides in front of everything, which the data service re-checks, and two outside systems that see opposite things: the sign-in service sees who you are and never your data, and the model provider sees the shape of your data and never who asked.

The first picture. Diagram 01 draws the platform as one box. Look at the four kinds of person entering through one browser, and at the two systems the platform calls outward.

System context: the Forma company app platform, the people who use it at their entry points, and the systems it calls (arrows: synchronous calls; protocol and authentication in brackets)

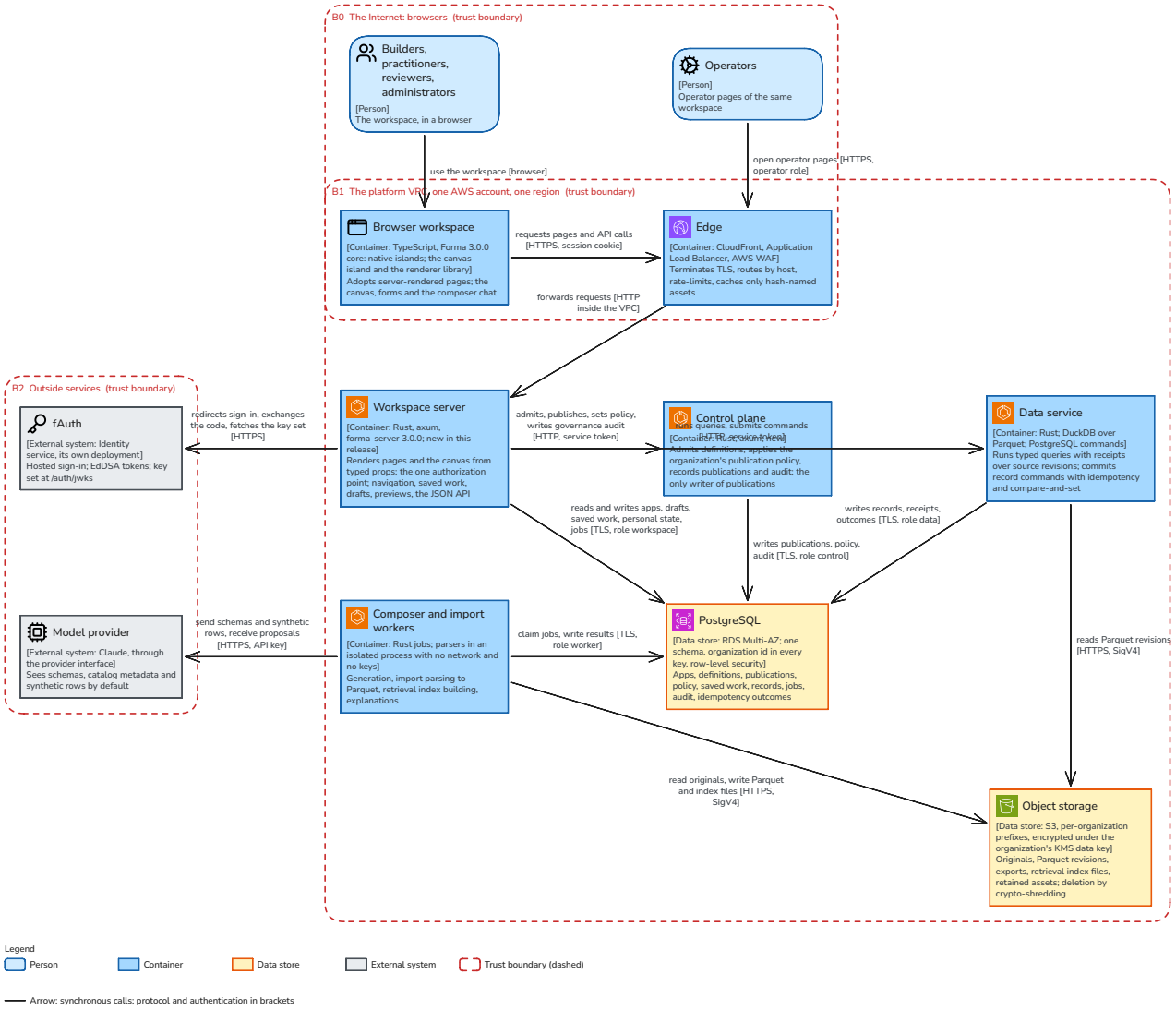


System context: the Forma company app platform, the people who use it at their entry points, and the systems it calls

What matters is that every person enters through one session and one decision point, and that the two external systems see opposite things: fAuth sees who signs in, never data; the model provider, Claude behind a provider interface, sees schemas, catalog metadata and synthetic rows, never who asked. It simplifies: sign-in is a browser redirection drawn as the platform's call, the operator's just-in-time role is drawn as the platform's own, and assistant hosts and connectors are absent because they are non-goals (section 3.2).

The second picture. Diagram 02 zooms into that box. Look for two paths through one gate: the interactive path down the left, and the background path along the bottom.

Containers: inside the platform box of diagram 01; the deployable roles, the stores and the trust boundaries, and who calls whom (key management is on the deployment diagram) (arrows: synchronous calls; protocol and authentication in brackets)



Containers: inside the platform box of diagram 01; the deployable roles, the stores and the trust boundaries, and who calls whom

The interactive path: the browser workspace, running the released framework, takes over pages the workspace server has already rendered from typed values and calls a JSON API through the edge; the workspace server is the one place that decides, and the only caller of the other two roles, each of which re-checks the member it is told about. The background path: workers claim jobs from the database, read uploaded originals, write column files and search indexes to object storage, and alone talk to the model provider; the file parser runs as a sealed process inside a worker, with no network and no keys of its own. Two rules of ownership hold everything together: the control plane alone writes publication records, and the data service alone reads the evidence files and writes records; published apps are served from the workspace server's own reads of the publication rows, so the control plane is off the member's path. The diagram leaves out the key-management service, drawn on the record's deployment diagram, and monitoring, backups, the upload stream, the export download and the operator pages.

The scenario, told as a story. The record walks eight steps on the demo's synthetic fixtures, as if the platform existed; "built" marks what runs today on Forma 3.0.0 or fAuth, and "proposed" marks this design.

1. Maya signs in at Northstar's branded, invite-only fAuth page (built). The workspace server verifies the signed token offline, takes the organization from the token's tenant field, mints a session, and shows the spaces her membership permits (proposed).
2. She drops the connection-events CSV and the inventory spreadsheet onto the canvas; the sealed parser writes revision 1 of each as an immutable column file under Northstar's key. Maya confirms the mappings: `bytes` is per event, timestamps are UTC, an exact inventory entry wins over a range, and a connection is counted once.
3. She asks for "build-worker-17's outbound bytes for the ten minutes from 10:00, by destination, matched against the inventory". The composer worker first searches what the organization already has and labels what it found; then it sends the provider the schemas and synthetic rows, never the evidence, and gets back a typed definition of six linked views, which the control plane admits and the canvas previews (built: the rendering; proposed: the rest). She adds a transfer-rate measure over the full ten minutes and an observation form, each a patch with a readable diff.
4. Sarah, the reviewer, approves the diff and the preview; her own session signs the content digest, and the control plane writes the publication record under Northstar's independent-review policy, which had refused Maya's attempt to publish her own work.
5. Maya opens the app and selects `build-worker-17` in the window from 10:00 to 10:10. Six typed queries come back, each with a receipt: 1,140,000 bytes in total, 240,000 matched, 900,000 unmatched to `198.51.100.7`, and 114,000 bytes a minute over the ten-minute denominator. She records the finding and saves the case, which pins publication 1.
6. She shares the case with Jordan, a grant separate from access to the app; he reopens the same work and sees the same records and receipts. For another host he starts new work, 210,000 bytes and 30,000 unmatched, nothing of Maya's.
7. In the Finance space, Spend Review forms through the same path, and a decision is submitted as a record command; chapter 5 tells it, with the lost response, the retry and the stale edit.
8. Maya returns to the Security space: her case, filter and layout come back, and a Finance query she left running answers late with an old epoch and is discarded. Separately, the canary, a second synthetic organization, replays every call with Northstar's identifiers and gets nothing.

Does it fit, warm? Thirty concurrent users at one interaction every 10 seconds are 3 interactions a second and 18 query operations a second over six linked views; at the busiest moment, every one of 100 seats active, 60 operations a second. At an assumed 100 ms per warm operation, about 6 are in flight at once, so one data node running at most 8 queries at a time, at most 2 per organization, carries the design point, and a six-view refresh runs as three waves of two, about 300 ms; a second data node exists for availability, not load. Record commits follow from one record a minute per active practitioner: 0.5 a second on average, tested at 2 a second on one record. Eight job slots, three for imports and five for generation,

one of each per organization, so at most five organizations generate at once and a sixth waits in a visible queue.

And cold? That fit check is warm. A query whose source revision is not yet on the node fetches and decrypts it first, and the record gives each operation a deadline of 1.8 seconds, so a cold six-view refresh is three waves of up to 1.8 seconds each, which misses the 2-second target. Whether a cold aggregate over a 1-million-row revision actually fits inside 1.8 seconds is unmeasured, and it is the first thing the spike measures; the record carries the question openly, two data tasks with a cap of 2 against a third task and a cap of 6, settled by load, with pre-aggregation at publication as the fallback (sections 5.2, 13.2). Until then the design point is carried warm and assumed cold.

The trade-off. Four roles from one binary means eight containers for one founder to run; the record accepts that because each role boundary is a security boundary, a credential that cannot write what another writes (section 12.2).

What to check first. The status labels: what is built is the rendering of a four-kind canvas on Forma 3.0.0 as shipped and fAuth at its pin; everything else in the two pictures is proposed (section 5).

3. Why the second app is the proof

The question. What shows this is a platform and not one app with a chat box bolted on?

The answer. Spend Review, a finance queue with a decision form, comes through the same authoring path and the same build as Northstar's security investigation, with no platform code changed between them; the record makes that the goal the release is judged on. It is an engineering proof of generality, not evidence that anyone wants the second app; the record keeps those apart.

How it works. Goal G1 is "two apps, one build": done when both apps come through the supported authoring path on one build digest and no platform code branches on a tenant or space name; requirement N9 adds zero commits and zero deploys between them. The record's change scenario is that Spend Review is built from form, status and detail components Northstar never used, and the revision and deployment logs show no change (section 4.2).

What makes this structural is the representation, decision D1. An app is a typed definition: views drawn from a closed vocabulary, queries from a bounded grammar, record schemas and declared commands. The platform's own compiled components render it; the model proposes data, never code, markup or grants. Abridged, this is what the model proposes for Maya's app before she adds the transfer-rate measure:

```
{ "app": "outbound-investigation", "schema": 1,
  "inputs": { "events": "connection-events@rev1", "inventory": "inventory@rev1" },
  "views": [
    { "id": "total", "kind": "summary", "measure": { "sum": "events.bytes" } },
    { "id": "destinations", "kind": "table", "group": "events.destination", "lookup": {
"inventory": "cidr" } },
    { "id": "timeline", "kind": "timeline", "x": "events.time", "y": { "sum": "events.bytes"
} } ],
  "filters": [ { "id": "destination", "applies": ["total", "destinations", "timeline"] } ],
  "records": { "observation": { "fields": { "status": ["open", "noted"], "note": "text<2000"
} } },
  "commands": [ { "id": "annotate", "record": "observation", "transitions": { "open":
["noted"] } } ] }
```

Everything in it names a kind, a measure, a filter or a record. There is no URL, no path, no identifier of another organization and no grant in the grammar, so a proposal that tries to carry one fails validation

whole before it is stored. The matching rule from chapter 1 is not in the definition either: it is confirmed at import and fixed into the revision the definition names, so the receipt pins it by pinning the revision. Admission at the control plane then re-checks the schema version, every query against the grammar, a complexity budget of at most 24 views, 8 operations a view, 256 KiB of definition and 500 rows a page, and that every component kind and query operator names a registered version; an unknown one fails, and nothing falls back (section 6.2).

Goal G6, honest AI, follows. Every creation is labelled retrieval, adaptation or generation, so that a demo moment is never mistaken for novelty; on a frozen set of ten supported and five adversarial requests, every admitted result must validate and every unsupported one must abstain, naming the missing capability. The model's five tools hand it identifiers only, the query tool returns a receipt id and a row count, never rows, and an explanation is built from the receipt and whatever fields the organization's policy names. When the provider is down, navigation, published queries and forms keep working while generation fails visibly. And to say it plainly: the demo's generation moment calls the live model, with at most two attempts, and fails visibly on stage if both miss; only the spike, the measurement that precedes the demo, uses a labelled fixture, and fixtures never pass the gate (sections 5.3 D8, 11.1).

The cost of the closed vocabulary. The renderer library ships nine kinds: summary, table, detail, timeline, chart, filter, form, note and status. A widget it cannot express is an engineering release, not a prompt. The prototype measured four of the nine on one canvas, at 27,674 bytes of framework code compressed on the route, against 28,100, which is Forma's own route ceiling: the compiler refuses a route whose bundled framework code exceeds it (its diagnostic BUILD-819). That leaves 426 bytes, so a fifth kind that imports the framework's resource loader or its HTML runtime waits on a framework budget ticket or a measurement. The record says plainly that the demo's generation moment shows assembly, not a bespoke widget, and that the custom-widget tier, which is exactly design A's markup path, waits for a customer to ask (section 12.1). Design A's markup path loses here: it needs five priority-one framework tickets, two of them security-critical, and its runtime alone weighs 28,337 bytes compressed, over the ceiling, while the typed definition needs no sanitizer, no translator and no sealed preview frame (section 12.2).

The proof on stage. The Northstar demo is recorded in six acts: enter the workspace, bring evidence, form and transform, publish and run, share and reuse, a different app; chapter 2's eight steps are the same path at the record's grain. The demo is a slice of this record, in one process, with synthetic data and the same contracts, so that it grows into the platform without a rewrite (section 11.1).

The trade-off. Generality over expressiveness: two apps through one path with no deploy between is a proof a closed vocabulary makes structural, and the same closure is what disappoints a team whose app needs the tenth kind.

What to check first. D1 and the spike, where the record sends its own reviewer first (section 13.1): run the fifteen requests and watch how many abstain and how each abstention is worded.

4. What keeps organizations apart

The question. Ten companies' evidence sits in one database and one bucket; how does none of it reach another company?

The answer. Every key begins with the organization's id, taken only from the verified sign-in token; the database filters every table by it with row-level security; each organization's files are encrypted under its own key; and a second synthetic organization, the canary, replays every call with Northstar's identifiers on every release and must get nothing back.

How it works. Decision D3 is pooled tenancy: shared tables and shared buckets, with the organization's id in every key and one data key per organization; a separate pool, a cell, for any organization that would later starve the shared one. It is a one-way door, because adding ids later would mean migrating every

store. The alternative was a silo, a database and a key per organization: ten databases now and a hundred at ten times the design point, each run alone by one operator (section 12.2).

The pool has two independent walls. Against a query that forgets its organization: the id comes from the verified token, and the row policy in the database filters anyway. Against a role that sets the wrong id: the sign-in adapter is the id's only source, no request body, header or path is ever believed, and the canary suite replays every route. The record is blunt about the price: a bug in a row policy is a cross-organization bug, so the suite runs on every release. Denied, another organization's identifier and an absent identifier all answer 404, so that existence does not leak. The canary is goal G5 and the one test the release cannot fail: it is denied before data or even metadata on every call, a removed member's open display shows denied within 5 seconds, and any canary failure during a rollout rolls the release back on its own (section 11.1).

The model provider. By default the composer sees schemas, catalog metadata and 20 synthetic rows; real values reach the provider only under a policy the organization's administrator writes, naming the fields and the provider, and a request carrying a field the policy does not name is refused before it leaves (sections 6.4, 10.5).

The three threats an investor should read. Section 10.4 of the record rates seven threats of high impact, and appendix D twenty-two more; these three carry the argument.

- **TH1, reading across organizations.** A member of any organization with a guessed or copied identifier, or a request handler that omits its scope, could read or act on Northstar's work. The organization comes from the session only, row-level security covers every table, files live under per-organization prefixes and key contexts, and the canary suite runs each release; high on both likelihood and impact before controls, low after.
- **TH2, a forged sign-in token.** Whoever holds fAuth's signing key could mint a token for any person in any organization. A session is minted only for a person who was invited, an unknown key id is refetched at most once a minute and then refused, and mints are audited; the move of fAuth's key into a hardware-backed service is a separate ticket, so the residual stays medium until it lands.
- **TH7, the operator.** The operator, or whoever holds their credentials, could read or delete an organization's data or alter the record of doing so. The operator sees no organization data; reading it needs a one-hour grant the organization's own administrator approves; the emergency path is a sealed role behind a hardware key whose use alerts a named outside reviewer first; and deletion, disabling a key, mass sign-out and permission changes take two people. Low likelihood, high impact, and the residual stays medium while there is one person.

Five accepted risks a customer would be surprised by, in the record's words, shorter; the founder owns eleven in all (section 10.7).

- One operator: platform-wide destructive actions wait 24 hours with notice instead of a second person, and until a named outside reader exists, breakglass alerts the founder alone.
- Pooled roles: a compromised workspace server reads what any member may read, and a compromised data service every revision, because a silo is ten stacks for one operator.
- Plain HTTP inside the private network, between the load balancer and the services and between the services, because mutual TLS needs a certificate authority nobody runs.
- fAuth's signing and management keys sit with the same founder on one deployment until the ticket moves them.
- One region: losing it stops service until it returns, because the backups stay in it.

The trade-off. Pooled tenancy buys one schema to migrate and one stack to run, and pays with a test suite that must run on every release because one policy bug is every organization's bug.

What to check first. TH1 and the canary suite. And one thing before any real row: connection logs identify employees, so the first real organization needs a data protection impact assessment wherever the GDPR or a law like it reaches it (section 10.1).

5. What a number means

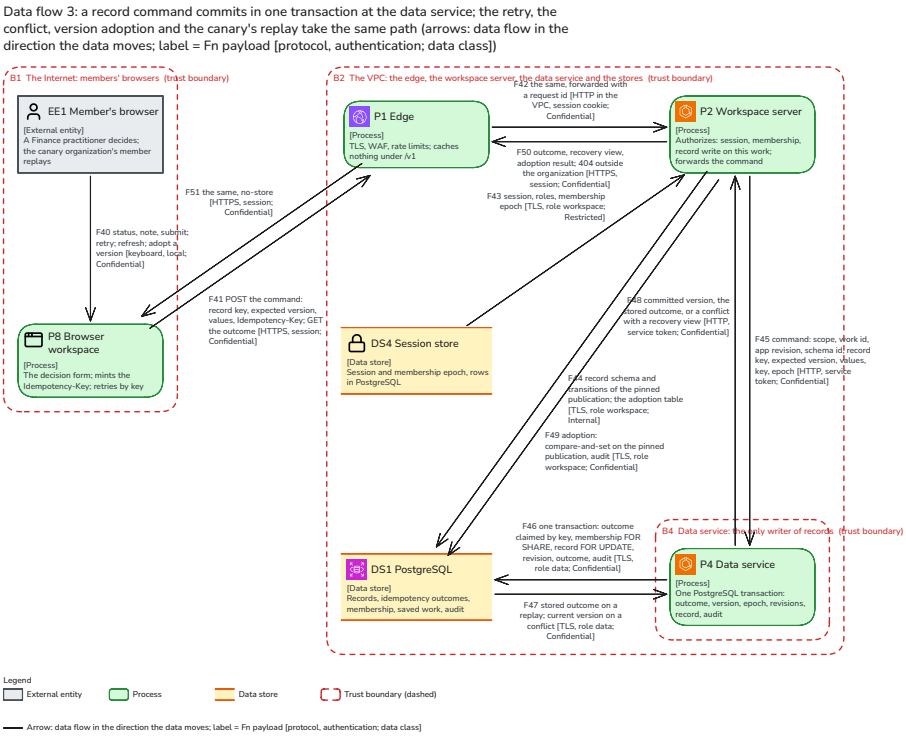
The question. When the app shows 1,140,000 bytes, why should anyone believe it, and when a decision is written back, why can it not be lost or doubled?

The answer. Because the number was computed on the server, over an immutable copy of the source and the full authorized input, and arrives with a receipt naming exactly what produced it; and because a decision is one database transaction, keyed so that a retry returns the same outcome and a stale edit gets a conflict instead of silently winning.

Receipts. A receipt pins the definition's digest; the versions of the query operators, the sum, the per-minute rate, the network-range lookup, which are versioned software and have nothing to do with the human operator of chapter 4; the digests of the source revisions; the record snapshot; the parameters; the policy epoch; and the digest of the result. Identical receipts come from a cache keyed by that hash, and every cached result is re-authorized before release, so the cache buys latency and can never be stale. Goal G3 says what "correct" means: totals cover the full authorized input before pagination, time windows are half-open, money is integer cents and a sum across currencies is refused, and a failed calculation is an error, never a zero (sections 3.1, 6.3).

Immutable sources and two stores, decision D4. An import is an immutable revision, the original and a column-file copy named by digest and encrypted under the organization's key, with the mappings the importer confirmed fixed into it; a receipt therefore pins a revision forever, and a deleted source keeps its receipts rather than substituting a newer file. Evidence lives as those column files, queried by an embedded analytical engine, DuckDB, through the typed grammar; records, saved work and publications live in PostgreSQL; one data service owns both. The reason is a sentence: evidence never changes and records always do; a million rows of a source cannot sit in the database every role uses, and a form needs a transaction. It is a one-way door, because receipts and saved work are made of it. A query past its 1.8-second budget answers "narrow the query", never a partial number (section 6.3).

The record command, decision D5. Diagram 06 follows one such command, Spend Review's decision, from the form to the data service's transaction and back; look at the single arrow into the database, where everything that matters happens.



Data flow 3: a record command commits in one transaction at the data service; the retry, the conflict, version adoption and the canary's replay take the same path

What matters is that the retry lookup, the version check, the membership check and the writes all happen inside one transaction the data service owns, so that no cache and no check at the HTTP entry is the guarantee. The diagram simplifies: the administrator's revocation is drawn in another flow, and the canary is just a member signed in to another organization. One arrow deserves a word: version adoption, moving a saved case onto a newer publication, is drawn from the workspace server, because a case is saved work, which the workspace server owns, not a record; the data service holds the case's row shared during a command so that the two cannot interleave.

The story, simplest case first. A finance practitioner opens the review queue of a 64,000-cent spending table filtered at 20,000 cents, where a single expense, EXP-002 at 45,000 cents, sits above the threshold; the fixture is small on purpose. She submits "needs context" with a note, expecting version 0 because the record does not exist yet; the browser mints the retry key `ik-1`. The workspace server checks that this member may write records in this work and that the fields and transition are ones the published record schema declares, and forwards the command. The data service runs one transaction, and version 1 comes back; the queue shows "needs context" where it showed "Pending"; the 64,000-cent total is unchanged, and no payment happened. Then the two cases that break spreadsheets. The response is lost and the browser retries with the same key: the stored outcome is found before any version check, so the same outcome returns and exactly one mutation exists. A colleague with a stale screen submits "reviewed" still expecting version 0 under a new key: the record is at version 1, so the transaction rolls back and the colleague gets a conflict with an authorized view of the current state, never a silent overwrite. After a refresh, "reviewed" expecting version 1 commits version 2.

Inside the transaction, in three sentences. The data service first re-checks the member it was told about, then claims the retry key under a uniqueness rule, so a duplicate waits and reads the stored outcome before any version is compared. It then holds the membership and case rows shared and the record row exclusively, so a revocation or an adoption, which take those rows exclusively, and a second writer,

serialise with it in one order or the other. Only then does it write the new record revision, the outcome and the audit event; one busy record sustains about 50 commits a second at an assumed 20 ms commit, against 2 a second tested (section 8).

Three epochs keep a screen honest. A filter epoch in the browser increases on every filter change, and any response carrying an older one is dropped, so the six linked views never disagree. A membership epoch increases on every change of membership, is read on every request, and the canvas polls it every 5 seconds, so a removed member's open display goes dark within the 5-second target. A policy epoch increases when the organization's policy changes, and the receipt carries it, so a result computed under an older policy is never released from the cache.

Saved work and the audit. A saved case pins the publication it was started on and moves to a newer one only when the practitioner adopts it explicitly, after seeing the changes; rolling an app back reverses no record (section 7.3). Every governance action, operator access and record command is an insert-only audit row, hash-chained and anchored to write-once storage every 15 minutes; each night a sample of cited receipts is recomputed, and a mismatch quarantines the revision (section 11.4).

The trade-off. Two engines to tune, two backup paths, and the unmeasured cold aggregate of chapter 2; the fallback, aggregates computed at publication, changes the engine's plan, not the grammar, which is why D4 is one-way and the choice of engine is not.

What to check first. The spike's cold-aggregate measurement, and goal G4's two tests: the retry that yields one mutation and the two concurrent edits that yield one commit and one visible conflict.

6. What it costs and how one founder runs it

The question. What does it cost to run, and can one person operate it?

The answer. About $1,150 a month at the design point, $115 an organization, and about $4,500 at ten times the load, $45 an organization, with the data service as the driver; one operator runs it on managed services, with one binary, six paging alerts and a release that rolls itself back on a single metric. The record says how; how many hours a week it is, it does not say.

Why more apps stay cheap. A new app is an admitted definition, a database row rendered by components compiled once: it needs no build, no runtime and no engine of its own, so adding an app adds no server to run. Pages render on the server in Rust from those compiled modules, with no JavaScript engine there at all. The spike measures the render rate and the memory per render, so the pitch will carry a measured number, not an analogy.

The cost. The record prices every line at list in one AWS region, assumed until the founder names one, and verified against vendor documentation at the gate. Compute on Fargate is the largest line at $565, of which the data service is $340, and that is the driver. The database, Multi-AZ, is $270. Logs and metrics are $100; two NAT gateways $75; the load balancer and web firewall $55; the model provider's tokens about $50, small because retrieval runs before generation and each organization gets one generation job at a time; keys $18; storage, backups and secrets $5, $5 and $3; content delivery and egress sit inside their free allowances at $0. That is the $1,150. Growth is sublinear, because the fixed lines amortise: $115 an organization at ten organizations becomes $45 at a hundred. The provider ceiling is stated too: five generation slots busy all month would cost about $26,000, far above the point where a per-organization budget stops generation; with no customer budget known, the requirement is the meter, every cost attributed to each organization monthly (section 11.2). At ten times, 600 operations a second at the busiest moment need eight data tasks plus one for a zone loss, and the database stays one primary grown one size with a read replica; growth is adding tasks with no contract or code change (section 4.2).

The service levels. The record refuses a contractual availability figure until there is hosting, a support owner and failure measurements; its internal objective for the path a published app takes is 99.5% of valid

requests a month, 3.65 hours, and the vendors' own commitments clear it only if no full restore happens in a month, since one at its 4-hour objective would exceed the budget by 21 minutes. Latency is measured in the browser: the six-view refresh at 2 seconds at the 99th percentile, and revocation reaching an open display within 5 seconds, which is never traded (section 11.3).

How one person runs it. One program is one release, deployed to the called roles first, so a new caller never meets an old callee. Each role runs one task in each of two zones, each sized to carry the design point alone, so losing a zone needs no scaling action. A release goes out in three waves, a tenth of requests plus all of the canary's, then half, then all, each baked for 30 minutes and 1,000 requests under the canary's suite. One metric alone rolls a wave back: a canary failure, which is any error, any cross-organization hit, or a receipt whose result differs from the one recorded for the same key. Six conditions page, among them any canary failure and an audit write failure; the rest are tickets. Failures have a named response: the database fails over to its synchronous standby in 60 to 120 seconds with no acknowledged write lost; with fAuth down nobody new signs in while live sessions work for up to 12 hours; with the provider down generation answers "unavailable" while published apps keep working; nothing fails open. A quarterly drill restores everything into an isolated account and passes when Maya's case reopens with 1,140,000 bytes recomputed and the canary suite passes, inside 4 hours (sections 11.3, 11.4).

From the demo to the platform. The demo runs as one process on one host with synthetic data, but on the managed database, object storage and key service from its first row, so of the record's five migration steps, each with a bridge and a way back, the first three are true at birth. Two remain: one process becomes four services, with both running, the same request types in process and over the network, and the canary routed first; and the edge goes in front, with the host's direct name kept as the way back (section 11.1).

The trade-off. The second data task, half of the $340 that dominates the bill, buys availability, not load. One region means that losing it stops service until it returns, because the backups stay in it. And the whole chapter assumes one managed browser offering, one region, one operator, membership managed by hand and a named owner before any live data; if that is wrong, the region, the providers, the team and the support hours become open questions (section 3.3, A8).

What to check first. The price assumptions, which the gate verified against vendor documentation but which move with the region and the provider tier the founder has not yet chosen (section 13.2), and the restore drill's pass condition.

7. What we do not know yet and what to do first

The question. What could sink this, and what is the first thing to do?

The answer. Four risks are rated high: no customer, a framework with 426 bytes of headroom, a sign-in service whose key sits with the founder, and one founder shipping every release; the first thing to do is a spike that runs the whole path end to end and measures the two numbers the design assumes.

The high risks, in the record's words, shorter.

- **No customer, no baseline (RK1).** The seventh goal, cheaper than the spreadsheet, has no mitigation here; the demo is the first test. If it happens: a well-made platform with no organization.
- **The stack (RK2).** The canvas route has 426 bytes of headroom under the 28,100-byte ceiling, and two framework tickets are assumed done but not built: FORMA-232, route size budgets in the framework, and FORMA-246, the composition fixture kept as a conformance test. Forma 3.0.0 itself was published on 2026-10-20. The route is measured on each release and the budget ticket gates additions. If it happens: a fifth component waits for a framework release, and a moved framework contract reopens the ruling behind D1. D1 is one-way because every stored app is written in it; a contract that moves before any organization has stored one reopens the choice, which is why the spike comes first.

- **fAuth (RK3).** The live service is behind the pinned version, the platform's client row is made by hand, and its signing key is with the founder. The mitigations are a redeploy at the pin, one client row per environment, and the ticket that moves the key into a hardware-backed service. If it happens: a minimal platform login behind the same adapter.
- **One founder ships every release (RK4)**, and the founder's coding agent, the AI assistant that writes code in the same repositories, shares the release pipeline; the record forbids its identity to tag a release, edit a workflow or merge. Tags by the founder only, build actions pinned to commits, keyless signing, a bill of materials, deployment by digest. If it happens: hostile code in a signed build reaches every organization.

Three of the medium risks, in a line each: the engine's cold aggregate over a million rows misses 1.8 seconds, so cold views are slower and receipts unchanged; the closed vocabulary disappoints, so one app waits for a component and the pitch leans on receipts; the composer fails the fifteen-request suite in two attempts, so the demo falls back to retrieval of what the organization already has, labelled as such. Four more are in the record (section 13.1).

Before build, three questions. Does a cold aggregate fit inside 1.8 seconds, and does the real canvas route fit under 28,100 bytes with the renderer library imported? Both fit, or pre-aggregation and a framework budget; the spike decides. Which provider and tier? The fifteen-request suite decides. Which region? The priced one until the founder rules (section 13.2).

What exists, and the spike. What has been prototyped is the rendering: a four-kind canvas compiled as one island rendered 6,752 bytes of HTML with every fixture value present at first paint under a strict content-security policy, at 27,674 of 28,100 bytes compressed. Sign-in against fAuth works at its pin. Nothing else runs. The spike runs the chain end to end, admitted definition, authorized query, receipt, command, outcome, reopen, with the duplicate submission and the version conflict, and measures the cold aggregate over a 1-million-row revision against the 2-second refresh target and the real canvas route against the ceiling (section 5).

The one-way doors. Four decisions cannot be undone without migrating every organization, because they are what an organization's stored apps, evidence, keys and receipts are made of: D1, the typed definition, since every stored app is written in it; D2, the publication record, since it is the organization's evidence of what ran; D3, pooled tenancy, since ids added later would migrate every store; and D4, the two-store split, since receipts and saved work are made of it. The hosting region closes per organization once its data lives there. Everything else in the decision table reverses (section 5.3).

Still unknown. Whether anyone pays for governed reuse; how the model behaves on a real schema; what a real organization's retention should be. Future work, none of it a reason to accept the design: the custom-widget tier, apps inside an assistant, a connector, one bounded external action, cells, a second region (section 13.2).

What to check first. D1 and the spike. That is where the record sends its own reviewer, and the spike is the first measured answer to the two numbers this edition has had to call assumed.

8. The map to the record

Each chapter of this edition summarises these sections of `design.md`; the right-hand column says where the things it points at but does not reproduce live.

Chapter	Record sections	Not reproduced here
Summary in plain words	1; 5.3	the decision table D1 to D15, 5.3

Chapter	Record sections	Not reproduced here
1. What it is and why now	2; 3.1 (G7); 12.2	the glossary and references, appendix A and B
2. How it works	5; 5.1; 5.2; 6.1 to 6.6; 13.2	the components, 6; flows 1 and 2, 7.1 and 7.2
3. Why the second app is the proof	3.1 (G1, G6); 4.2 (N9); 5.3 (D1, D8); 6.2; 6.5; 11.1; 12.1; 12.2	the requirements with their tests, 4
4. What keeps organizations apart	3.1 (G5); 5.3 (D3); 6.4; 10.1; 10.4 (TH1, TH2, TH7); 10.5; 10.7; 12.2	the threat table, 10.4 and appendix D; trust levels, assets and the attack tree, 10.2 and 10.3; secrets and retention, 10.6 and 8
5. What a number means	3.1 (G3, G4); 5.3 (D4, D5, D14); 6.3; 7.3; 8; 11.4	the sequence diagram, 7.3; the API surface, 9
6. What it costs and how one founder runs it	3.3 (A8); 4.2 (N1, N2); 11.1; 11.2; 11.3; 11.4	the cost table with unit prices, 11.2; the failure modes table, 11.4; the dependency table, appendix C
7. What we do not know yet	5; 5.3; 13.1; 13.2	the alternatives, 12.2; abandoned ideas, 12.3